Experiment 1: Vector and Matrix Operations for Deep Learning

Aim:

To develop a bookstore recommendation system using customer preferences, genre-book relationships, cosine similarity, and a dense neural network layer, and visualize the results using a heatmap.

Algorithm:

1. Create a customer-genre preference matrix and a genre-book relationship matrix.
2. Calculate cosine similarity to identify customers with similar preferences.
3. Multiply the matrices to estimate customer preferences for different books.
4. Apply a dense neural network layer using weights and bias.
5. Display the recommendation scores.
6. Visualize the customer preferences using a heatmap.

```
import numpy as np

import matplotlib.pyplot as plt

customers = np.array([

    [5, 4, 0, 2, 1],

    [4, 5, 1, 2, 0],

    [0, 1, 5, 4, 3],

    [1, 0, 4, 5, 4],

    [3, 3, 2, 1, 4],

    [5, 2, 1, 3, 2]

], dtype=float)

books = np.array([

    [1, 0, 0, 1],

    [0, 1, 1, 0],

    [1, 1, 0, 0],

    [0, 0, 1, 1],

    [1, 0, 1, 0]

], dtype=float)

print("Customer Matrix:\n", customers)
```

```
print("\nBook Matrix:\n", books)
print("\nDot Product of first two customers:",
    np.dot(customers[0], customers[1]))
scores = np.dot(customers, books)
print("\nCustomer-Book Score Matrix:\n", scores)
print("\nTranspose of Customer Matrix:\n", customers.T)
print("\nElement-wise Addition:\n", customers + 1)
print("\nElement-wise Multiplication:\n", customers * customers)
square_matrix = np.array([[4, 7], [2, 6]], dtype=float)
print("\nInverse of Matrix:\n", np.linalg.inv(square_matrix))
norms = np.linalg.norm(customers, axis=1)
similarity = customers @ customers.T / np.outer(norms, norms)
np.fill_diagonal(similarity, 0)
i, j = np.unravel_index(np.argmax(similarity), similarity.shape)
print("\nCosine Similarity Matrix:\n", np.round(similarity, 3))
print("Most Similar Customers:", i + 1, "and", j + 1)
print("Similarity:", round(similarity[i, j], 3))
W = np.array([[0.2, 0.4], [0.5, 0.1]])
x = np.array([5.0, 4.0])
b = np.array([1.0, 2.0])
y = W @ x + b
print("\nDense Layer Output:", y)
plt.imshow(scores, cmap="YlGnBu", aspect="auto")
plt.colorbar(label="Recommendation Score")
plt.xlabel("Book")
plt.ylabel("Customer")
plt.title("Customer-Book Score Heatmap")
plt.xticks(range(4), ["Book 1", "Book 2", "Book 3", "Book 4"])
plt.yticks(range(6), ["C1", "C2", "C3", "C4", "C5", "C6"])
plt.tight_layout()
```

```
plt.show()
```

OUTPUT

```
Customer Matrix:
 [[5. 4. 0. 2. 1.]
 [4. 5. 1. 2. 0.]
 [0. 1. 5. 4. 3.]
 [1. 0. 4. 5. 4.]
 [3. 3. 2. 1. 4.]
 [5. 2. 1. 3. 2.]]

Book Matrix:
 [[1. 0. 0. 1.]
 [0. 1. 1. 0.]
 [1. 1. 0. 0.]
 [0. 0. 1. 1.]
 [1. 0. 1. 0.]]

Dot Product of first two customers: 44.0

Customer-Book Score Matrix:
 [[6. 4. 7. 7.]
 [5. 6. 7. 6.]
 [8. 6. 8. 4.]
 [9. 4. 9. 6.]
 [9. 5. 8. 4.]
 [8. 3. 7. 8.]]

Transpose of Customer Matrix:
 [[5. 4. 0. 1. 3. 5.]
 [4. 5. 1. 0. 3. 2.]
```

[0. 1. 5. 4. 2. 1.]

[2. 2. 4. 5. 1. 3.]

[1. 0. 3. 4. 4. 2.]]

Element-wise Addition:

[[6. 5. 1. 3. 2.]

[5. 6. 2. 3. 1.]

[1. 2. 6. 5. 4.]

[2. 1. 5. 6. 5.]

[4. 4. 3. 2. 5.]

[6. 3. 2. 4. 3.]]

Element-wise Multiplication:

[[25. 16. 0. 4. 1.]

[16. 25. 1. 4. 0.]

[0. 1. 25. 16. 9.]

[1. 0. 16. 25. 16.]

[9. 9. 4. 1. 16.]

[25. 4. 1. 9. 4.]]

Inverse of Matrix:

[[0.6 -0.7]

[-0.2 0.4]]

Cosine Similarity Matrix:

[[0. 0.957 0.31 0.368 0.779 0.922]

[0.957 0. 0.372 0.348 0.732 0.832]

[0.31 0.372 0. 0.956 0.65 0.534]

[0.368 0.348 0.956 0. 0.673 0.641]

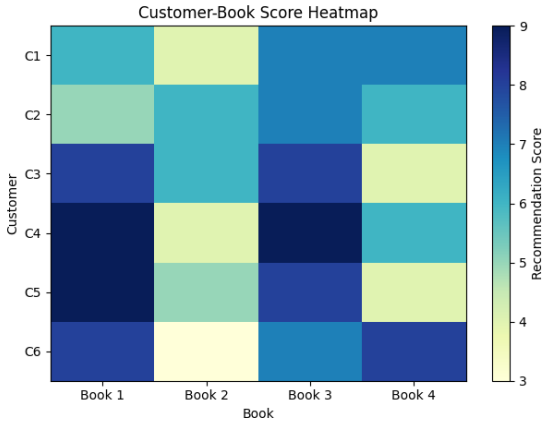
[0.779 0.732 0.65 0.673 0. 0.83]

[0.922 0.832 0.534 0.641 0.83 0.]]

Most Similar Customers: 1 and 2

Similarity: 0.957

Dense Layer Output: [3.6 4.9]



Result:
The bookstore recommendation system was implemented successfully, and book recommendations were generated based on customer preferences and similarity scores.

2. Covariance, Eigen Values, Eigen Vectors

Aim:
To calculate the covariance matrix, eigenvalues, and eigenvectors of a dataset and understand how they represent relationships and directions of variation in the data.

Algorithm:

1. Create or load a numerical dataset.
2. Calculate the mean of each feature.
3. Center the dataset by subtracting the feature means.
4. Calculate the covariance matrix.
5. Find the eigenvalues and eigenvectors of the covariance matrix.
6. Sort the eigenvalues in descending order and arrange their corresponding eigenvectors.
7. Interpret the eigenvalues and eigenvectors to identify the directions of maximum variation.

Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

np.random.seed(10)
```

```
n = 200
haemoglobin = np.random.normal(13, 1.5, n)
rbc = 4.5 + 0.25 * haemoglobin + np.random.normal(0, 0.3, n)
glucose = np.random.normal(110, 15, n)
cholesterol = 180 + 0.5 * glucose + np.random.normal(0, 12, n)
bmi = np.random.normal(25, 4, n)
df = pd.DataFrame({
    "Haemoglobin": haemoglobin,
    "RBC": rbc,
    "Glucose": glucose,
    "Cholesterol": cholesterol,
    "BMI": bmi
})
print("First Five Patient Records:\n", df.head().round(2))
X = (df - df.mean()) / df.std(ddof=0)
cov = X.cov()
corr = X.corr()
print("\nCovariance Matrix:\n", cov.round(3))
print("\nCorrelation Matrix:\n", corr.round(3))
values, vectors = np.linalg.eigh(corr)
order = np.argsort(values)[::-1]
values = values[order]
vectors = vectors[:, order]
variance = values / values.sum() * 100
print("\nEigenvalues:\n", values.round(4))
print("\nExplained Variance (%):\n", variance.round(2))
print("\nEigenvectors:\n", vectors.round(3))
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
im1 = ax[0].imshow(cov, cmap="coolwarm")
ax[0].set_title("Covariance Matrix")
```

```
ax[0].set_xticks(range(5), df.columns, rotation=45)
ax[0].set_yticks(range(5), df.columns)
fig.colorbar(im1, ax=ax[0])
im2 = ax[1].imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
ax[1].set_title("Correlation Matrix")
ax[1].set_xticks(range(5), df.columns, rotation=45)
ax[1].set_yticks(range(5), df.columns)
fig.colorbar(im2, ax=ax[1])
plt.tight_layout()
plt.show()
projected = X.to_numpy() @ vectors[:, :2]
plt.scatter(projected[:, 0], projected[:, 1], alpha=0.6)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Patient Data Projected onto First Two Components")
plt.show()
```

OUTPUT

First Five Patient Records:

	Haemoglobin	RBC	Glucose	Cholesterol	BMI
0	15.00	8.29	116.94	245.32	25.08
1	14.07	8.38	91.70	219.70	25.63
2	10.68	6.86	112.89	239.75	29.15
3	12.99	7.79	116.53	233.59	25.18
4	13.93	7.64	85.48	230.52	29.43

Covariance Matrix:

	Haemoglobin	RBC	Glucose	Cholesterol	BMI
Haemoglobin	1.005	0.793	-0.042	-0.056	-0.007
RBC	0.793	1.005	0.058	-0.049	-0.046
Glucose	-0.042	0.058	1.005	0.525	-0.046
Cholesterol	-0.056	-0.049	0.525	1.005	-0.022
BMI	-0.007	-0.046	-0.046	-0.022	1.005

Correlation Matrix:

	Haemoglobin	RBC	Glucose	Cholesterol	BMI
Haemoglobin	**1.000**	**0.789**	**-0.042**	**-0.056**	**-0.007**
RBC	**0.789**	**1.000**	**0.058**	**-0.049**	**-0.046**
Glucose	**-0.042**	**0.058**	**1.000**	**0.522**	**-0.046**
Cholesterol	**-0.056**	**-0.049**	**0.522**	**1.000**	**-0.022**
BMI	**-0.007**	**-0.046**	**-0.046**	**-0.022**	**1.000**

Eigenvalues:

[1.8005 1.523 0.9944 0.4822 0.1999]

Explained Variance (%):

[36.01 30.46 19.89 9.64 4.]

Eigenvectors:

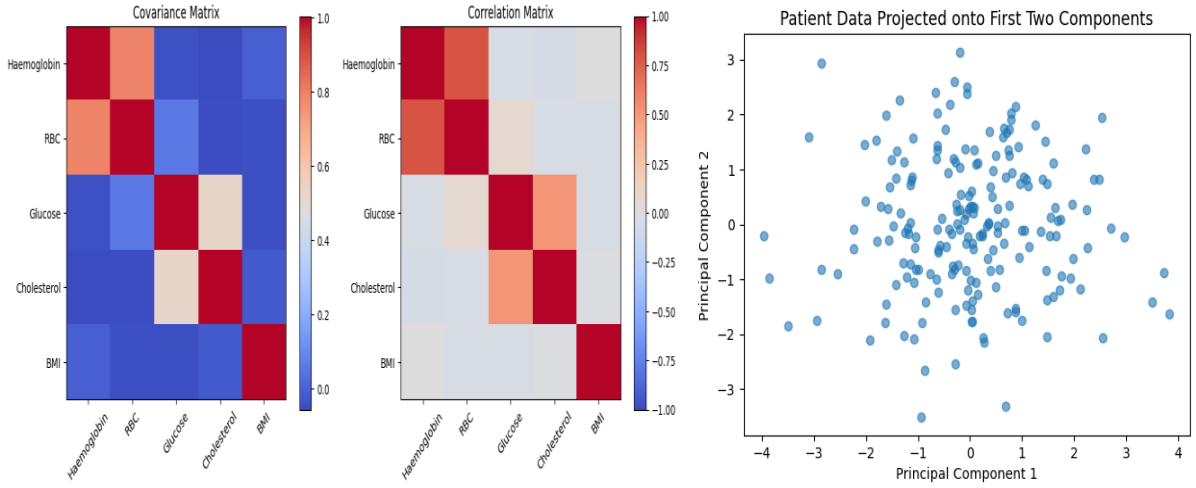
[[-0.7   0.073 -0.06 -0.147 0.693]

[-0.696 0.133 -0.018 0.083 -0.701]

[0.074 0.703 -0.05   0.69   0.143]

[0.139 0.686 -0.09 -0.702 -0.089]

[0.038 -0.104 -0.993 0.036 -0.028]]



Experiment 3: Probability Distributions and Bayes' Theorem

Aim:

To study probability distributions and apply Bayes' theorem to calculate conditional probabilities and update predictions using new evidence.

Algorithm:

1. Define the given events and their probabilities.
2. Calculate the joint and conditional probabilities.

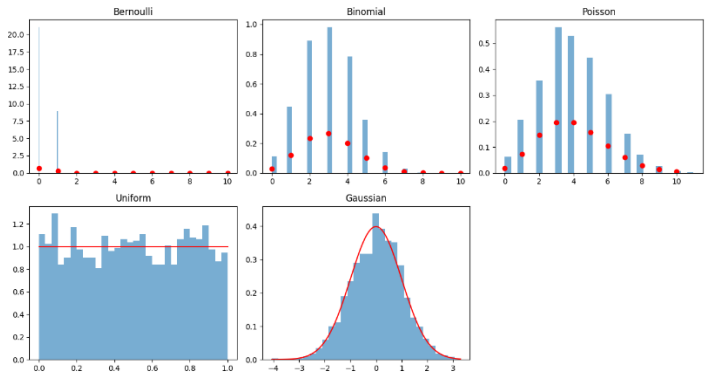
3. Apply Bayes' theorem to find the required probability.
4. Compare the calculated probabilities and interpret the results.

Code:

```
#3)
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import bernoulli, binom, poisson, uniform, norm
np.random.seed(42)
n = 100000
distributions = {
    "Bernoulli": bernoulli(0.3),
    "Binomial": binom(10, 0.3),
    "Poisson": poisson(4),
    "Uniform": uniform(0, 1),
    "Gaussian": norm(0, 1)}
fig, axes = plt.subplots(2, 3, figsize=(13, 7))
for ax, (name, dist) in zip(axes.flat, distributions.items()):
    samples = dist.rvs(size=2000)
    ax.hist(samples, bins=30, density=True, alpha=0.6)
    x = np.arange(0, 11) if name == "Binomial" else np.linspace(
        min(samples), max(samples), 200)
    if name in ["Bernoulli", "Binomial", "Poisson"]:
        x = np.arange(0, 11)
        ax.plot(x, dist.pmf(x), "ro")
    else:
        ax.plot(x, dist.pdf(x), "r-")
    ax.set_title(name)
axes.flat[-1].axis("off")
plt.tight_layout()
plt.show()
```

```
prevalence = 0.01
sensitivity = 0.95
specificity = 0.90
true_positive = sensitivity * prevalence
false_positive = (1 - specificity) * (1 - prevalence)
posterior = true_positive / (true_positive + false_positive)
print("Bayes' Theorem Result")
print("P(Disease | Positive) =", round(posterior, 4))
print("Percentage =", round(posterior * 100, 2), "%")
disease = np.random.rand(n) < prevalence
positive = np.where(
    disease,
    np.random.rand(n) < sensitivity,
    np.random.rand(n) < (1 - specificity))
empirical = disease[positive].mean()
print("\nSimulation Results")
print("Patients:", n)
print("Positive tests:", positive.sum())
print("Empirical probability:", round(empirical, 4))
print("Empirical percentage:", round(empirical * 100, 2), "%")
prev = np.linspace(0.001, 0.20, 200)
post = (sensitivity * prev) / (
    sensitivity * prev + (1 - specificity) * (1 - prev))
plt.plot(prev * 100, post * 100)
plt.xlabel("Disease Prevalence (%)")
plt.ylabel("P(Disease | Positive) (%)")
plt.title("Posterior Probability vs Prevalence")
plt.grid()
plt.show()
```

Output:



Bayes' Theorem Result

P(Disease | Positive) = 0.0876

Percentage = 8.76 %

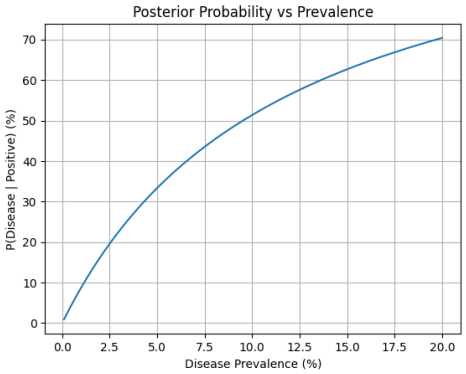
Simulation Results

Patients: 100000

Positive tests: 10887

Empirical probability: 0.0876

Empirical percentage: 8.76 %



Result:
Probability distributions and Bayes' theorem were implemented successfully, and the required conditional probabilities were calculated.

Experiment 4: Entropy, Cross-Entropy and KL Divergence

Aim:
To calculate entropy, cross-entropy, and Kullback–Leibler (KL) divergence and understand how they measure uncertainty and differences between probability distributions.

Algorithm:

1. Define the actual and predicted probability distributions.

2. Calculate entropy using the given probabilities.
3. Calculate cross-entropy between the actual and predicted distributions.
4. Calculate KL divergence and compare the distributions.
5. Interpret the calculated values.

Code:

```
#4)

import numpy as np
import matplotlib.pyplot as plt
from scipy.special import rel_entr
true = np.array([0.5, 0.3, 0.2])
model1 = np.array([0.6, 0.25, 0.15])
model2 = np.array([0.2, 0.3, 0.5])
def entropy(p):
    return -np.sum(p * np.log2(p))
def cross_entropy(p, q):
    return -np.sum(p * np.log2(q))
def kl_divergence(p, q):
    return np.sum(rel_entr(p, q)) / np.log(2)
H = entropy(true)
for name, model in [("Model 1", model1), ("Model 2", model2)]:
    CE = cross_entropy(true, model)
    KL = kl_divergence(true, model)
    print("\n", name)
    print("Cross-Entropy:", round(CE, 4))
    print("KL Divergence:", round(KL, 4))
    print("Entropy + KL:", round(H + KL, 4))
print("\nTrue Entropy:", round(H, 4))
print("KL(true || model1):", round(kl_divergence(true, model1), 4))
print("KL(model1 || true):", round(kl_divergence(model1, true), 4))
```

```
p = np.linspace(0.001, 0.999, 300)
y = 1
loss = -(y * np.log(p) + (1 - y) * np.log(1 - p))
plt.plot(p, loss)
plt.xlabel("Predicted Probability of Positive Class")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Binary Cross-Entropy Loss (Actual Class = 1)")
plt.grid()
plt.show()
```

Output:

Model 1

Cross-Entropy: 1.5159

KL Divergence: 0.0304

Entropy + KL: 1.5159

Model 2

Cross-Entropy: 1.8821

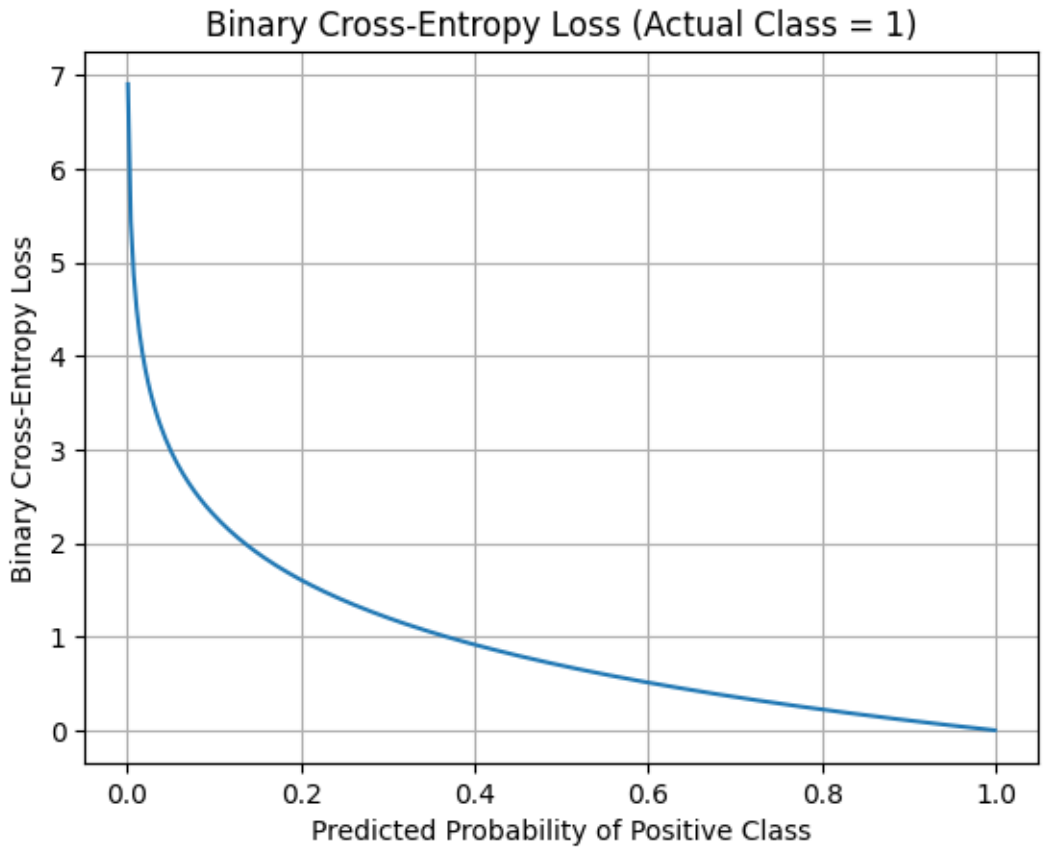
KL Divergence: 0.3966

Entropy + KL: 1.8821

True Entropy: 1.4855

KL(true || model1): 0.0304

KL(model1 || true): 0.0298



Result:
Entropy, cross-entropy, and KL divergence were calculated successfully to measure uncertainty and differences between probability distributions.

Experiment 5: Bias–Variance Trade-off and Overfitting

Aim:
To analyse bias, variance, underfitting, and overfitting by comparing model errors at different levels of model complexity.

Algorithm:

1. Generate or load a sample dataset.
2. Train models with different levels of complexity.
3. Calculate training and validation errors.
4. Plot the errors against model complexity.
5. Identify underfitting, overfitting, and the suitable model.

Code:

#5)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
np.random.seed(42)
X = np.sort(np.random.uniform(0, 10, 150)).reshape(-1, 1)
y = 2 * X[:, 0] + 0.5 * X[:, 0] ** 2 + np.sin(2 * X[:, 0])
y += np.random.normal(0, 3, 150)
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.25, random_state=42)
degrees = [1, 3, 9, 15]
train_errors = []
val_errors = []
grid = np.linspace(0, 10, 300).reshape(-1, 1)
plt.figure(figsize=(10, 6))
plt.scatter(X_train, y_train, label="Training Data", alpha=0.6)
plt.scatter(X_val, y_val, label="Validation Data", alpha=0.6)
for degree in degrees:
    model = make_pipeline(
        PolynomialFeatures(degree),
        LinearRegression())
    model.fit(X_train, y_train)
    train_pred = model.predict(X_train)
    val_pred = model.predict(X_val)
    train_errors.append(mean_squared_error(y_train, train_pred))
    val_errors.append(mean_squared_error(y_val, val_pred))
    plt.plot(grid, model.predict(grid), label=f"Degree {degree}")
```

```
    print("\nDegree:", degree)
    print("Training MSE:", round(train_errors[-1], 3))
    print("Validation MSE:", round(val_errors[-1], 3))
plt.xlabel("House Area")
plt.ylabel("House Price")
plt.title("Polynomial Regression: Underfitting and Overfitting")
plt.legend()
plt.grid()
plt.show()
plt.plot(degrees, train_errors, "o-", label="Training Error")
plt.plot(degrees, val_errors, "s-", label="Validation Error")
plt.xlabel("Polynomial Degree")
plt.ylabel("Mean Squared Error")
plt.title("Error vs Model Complexity")
plt.legend()
plt.grid()
plt.show()
best_degree = degrees[np.argmin(val_errors)]
print("\nBest Degree:", best_degree)
predictions = []
for seed in range(30):
    rng = np.random.default_rng(seed)
    indices = rng.integers(0, len(X_train), len(X_train))
    model = make_pipeline(
        PolynomialFeatures(best_degree),
        LinearRegression())
    model.fit(X_train[indices], y_train[indices])
    predictions.append(model.predict(grid))
predictions = np.array(predictions)
mean_prediction = predictions.mean(axis=0)
```

```
bias_squared = np.mean((mean_prediction - (
    2 * grid[:, 0] + 0.5 * grid[:, 0] ** 2
    + np.sin(2 * grid[:, 0])
)) ** 2)
variance = np.mean(np.var(predictions, axis=0))
print("Estimated Bias^2:", round(bias_squared, 3))
print("Estimated Variance:", round(variance, 3))
```

Output:

Degree: 1

Training MSE: 23.302

Validation MSE: 18.326

Degree: 3

Training MSE: 10.127

Validation MSE: 7.636

Degree: 9

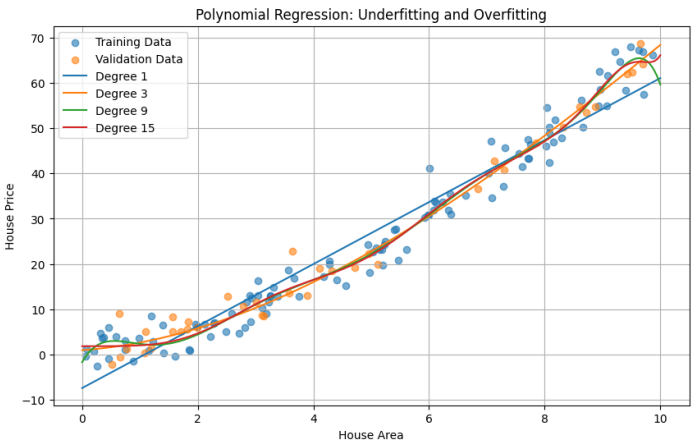
Training MSE: 9.198

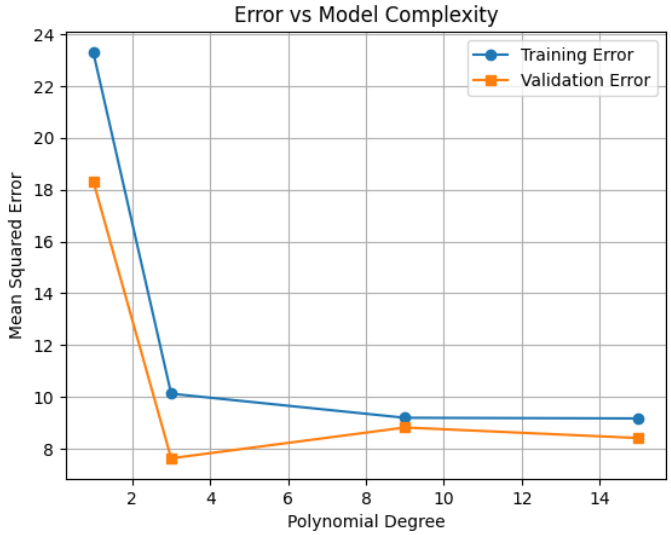
Validation MSE: 8.824

Degree: 15

Training MSE: 9.174

Validation MSE: 8.416





Result:
The bias–variance trade-off was analysed, and the relationship between model complexity, underfitting, and overfitting was identified.

Experiment 6: Gradient Descent, SGD, Momentum and Adam

Aim:
To implement and compare Gradient Descent, Stochastic Gradient Descent (SGD), Momentum, and Adam for minimizing a loss function.

Algorithm:

1. Define the model, parameters, and loss function.
2. Initialize the model parameters.
3. Calculate the gradients of the loss function.
4. Update parameters using Gradient Descent, SGD, Momentum, and Adam.
5. Compare the loss values and convergence of each optimizer.

Code:

#6)

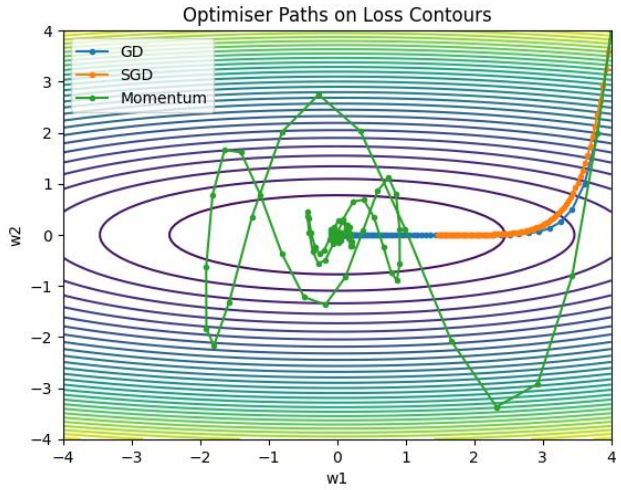
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
np.random.seed(42)
```

```
tf.random.set_seed(42)
def loss_fn(w):
    return 0.5 * (w[0] ** 2 + 10 * w[1] ** 2)
def gradient(w):
    return np.array([w[0], 10 * w[1]])
initial = np.array([4.0, 4.0])
optimizers = ["GD", "SGD", "Momentum"]
paths = {}
for name in optimizers:
    w = initial.copy()
    velocity = np.zeros(2)
    path = [w.copy()]
    for step in range(100):
        grad = gradient(w)
        if name == "GD":
            w -= 0.05 * grad
        elif name == "SGD":
            noisy_grad = grad + np.random.normal(0, 0.2, 2)
            w -= 0.01 * noisy_grad
        else:
            velocity = 0.9 * velocity + grad
            w -= 0.05 * velocity
        path.append(w.copy())
    paths[name] = np.array(path)
x = np.linspace(-4, 4, 100)
y = np.linspace(-4, 4, 100)
X, Y = np.meshgrid(x, y)
Z = 0.5 * (X ** 2 + 10 * Y ** 2)
plt.contour(X, Y, Z, levels=30)
for name, path in paths.items():
```

```
    plt.plot(path[:, 0], path[:, 1], marker=".", label=name)
plt.xlabel("w1")
plt.ylabel("w2")
plt.title("Optimiser Paths on Loss Contours")
plt.legend()
plt.show()
X_data = np.random.rand(500, 4).astype("float32")
y_data = (
    5 * X_data[:, 0] +
    3 * X_data[:, 1] +
    2 * X_data[:, 2] +
    X_data[:, 3]
).astype("float32")
X_train, X_val = X_data[:400], X_data[400:]
y_train, y_val = y_data[:400], y_data[400:]
optimizers = {
    "SGD": tf.keras.optimizers.SGD(learning_rate=0.01),
    "Momentum": tf.keras.optimizers.SGD(
        learning_rate=0.01,
        momentum=0.9
    ),
    "Adam": tf.keras.optimizers.Adam(learning_rate=0.01)
}
histories = {}
for name, optimizer in optimizers.items():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(4,)),
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dense(1)
    ])
```

```
    model.compile(optimizer=optimizer, loss="mse")
    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=50,
        batch_size=32,
        verbose=0)
    histories[name] = history.history["loss"]
    print(name)
    print("Initial Loss:", round(history.history["loss"][0], 4))
    print("Final Loss:", round(history.history["loss"][-1], 4))
    print("Final Validation Loss:",
          round(history.history["val_loss"][-1], 4))
for name, losses in histories.items():
    plt.plot(losses, label=name)
plt.xlabel("Epoch")
plt.ylabel("Training MSE Loss")
plt.title("Optimiser Comparison")
plt.legend()
plt.grid()
plt.show()
```

Output:



SGD

Initial Loss: 15.6359

Final Loss: 0.0018

Final Validation Loss: 0.0012

Momentum

Initial Loss: 11.6571

Final Loss: 0.003

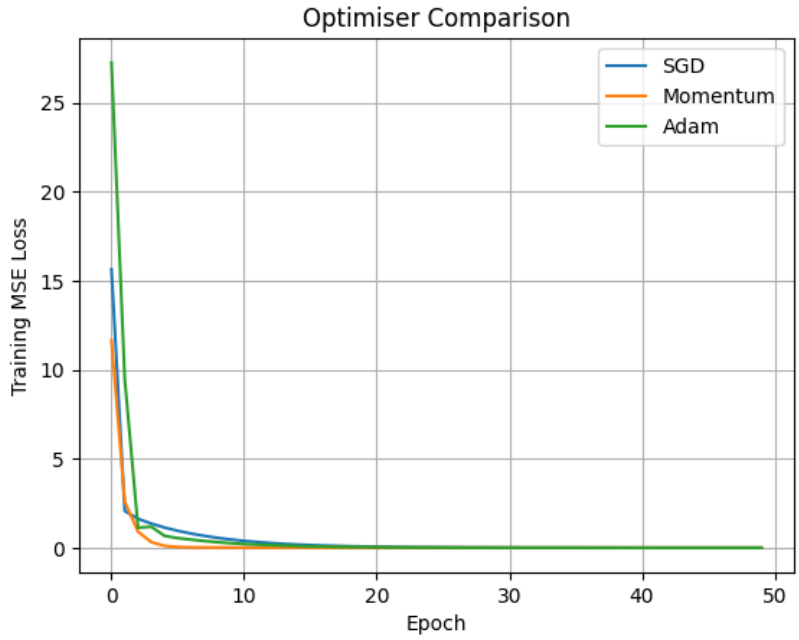
Final Validation Loss: 0.0008

Adam

Initial Loss: 27.2465

Final Loss: 0.0004

Final Validation Loss: 0.0004



Result:
The four optimization methods were implemented and compared based on loss reduction and convergence behaviour.

Experiment 7: Perceptron and Forward Propagation

Aim:
To implement a perceptron for binary classification, study forward propagation, and understand the limitations of a single-layer perceptron using linearly separable data and the XOR problem.

Algorithm:

1. Create a dataset with input features and target labels.
2. Initialize weights and bias.
3. Calculate the weighted sum of inputs.
4. Apply an activation function to predict the output.
5. Update weights and bias based on prediction errors.
6. Test the model using linearly separable data and XOR inputs.

Code:

```
#7) Perceptron and Forward Propagation


import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
def step(z):
    return (z >= 0).astype(int)
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))
def relu(z):
    return np.maximum(0, z)
# Task 1: Weighted sum, bias and step activation
x = np.array([0.7, 0.4])
w = np.array([0.8, -0.5])
b = 0.1
z = np.dot(w, x) + b
print("Weighted Sum:", round(z, 3))
print("Step Output:", step(np.array(z)).item())
print("Sigmoid Output:", round(sigmoid(z).item(), 4))
print("ReLU Output:", round(relu(z).item(), 3))
# Task 2: Generate 100 illustrative loan applications
X = np.random.rand(100, 2)
```

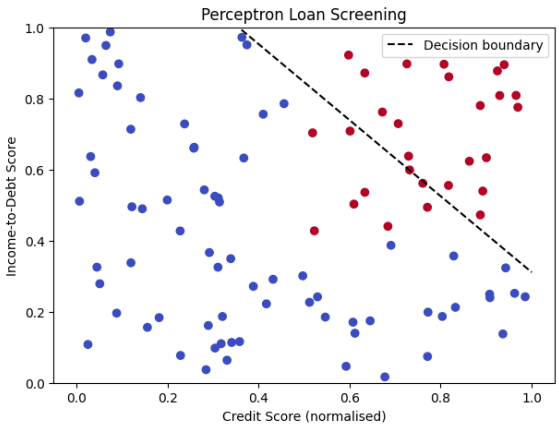
```
y = ((X[:, 0] >= 0.5) & (X[:, 1] >= 0.4)).astype(int)
weights = np.zeros(2)
bias = 0.0
lr = 0.1
epochs = 20
updates = []
for epoch in range(epochs):
    mistakes = 0
    for xi, target in zip(X, y):
        prediction = step(np.array(np.dot(weights, xi) + bias)).item()
        error = target - prediction
        weights += lr * error * xi
        bias += lr * error
        mistakes += int(error != 0)
    updates.append(mistakes)
    print("Epoch", epoch + 1, "Errors:", mistakes,
          "Weights:", np.round(weights, 3),
          "Bias:", round(bias, 3))
# Task 3: Decision boundary
plt.figure(figsize=(7, 5))
plt.scatter(X[:, 0], X[:, 1], c=y, cmap="coolwarm")
xx = np.linspace(0, 1, 100)
if abs(weights[1]) > 1e-10:
    yy = -(weights[0] * xx + bias) / weights[1]
    plt.plot(xx, yy, "k--", label="Decision boundary")
plt.xlabel("Credit Score (normalised)")
plt.ylabel("Income-to-Debt Score")
plt.title("Perceptron Loan Screening")
plt.ylim(0, 1)
plt.legend()
```

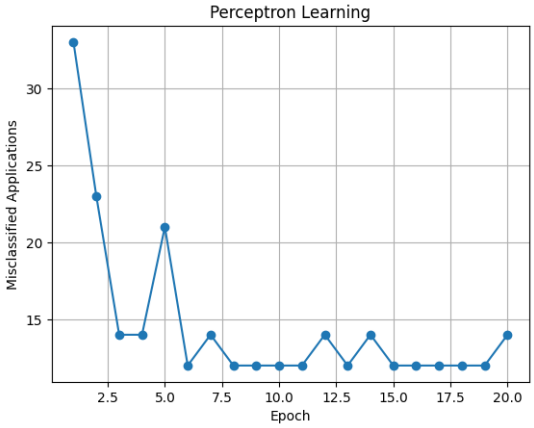
```
plt.show()
plt.plot(range(1, epochs + 1), updates, marker="o")
plt.xlabel("Epoch")
plt.ylabel("Misclassified Applications")
plt.title("Perceptron Learning")
plt.grid()
plt.show()
# Task 4: XOR
XOR_X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
XOR_y = np.array([0, 1, 1, 0])
w_xor = np.zeros(2)
b_xor = 0.0
for epoch in range(20):
    for xi, target in zip(XOR_X, XOR_y):
        prediction = step(np.array(np.dot(w_xor, xi) + b_xor)).item()
        error = target - prediction
        w_xor += 0.1 * error * xi
        b_xor += 0.1 * error
predictions = step(XOR_X @ w_xor + b_xor)
print("\nXOR Actual:", XOR_y)
print("XOR Predicted:", predictions)
print("XOR Correct:", np.sum(predictions == XOR_y), "out of 4")
print("XOR converged:", np.array_equal(predictions, XOR_y))
# Task 5: Activation comparison
z_values = np.linspace(-5, 5, 200)
plt.plot(z_values, (z_values >= 0).astype(int), label="Step")
plt.plot(z_values, sigmoid(z_values), label="Sigmoid")
plt.plot(z_values, relu(z_values), label="ReLU")
plt.xlabel("Input z")
plt.ylabel("Activation Output")
```

```
plt.title("Activation Function Comparison")
plt.legend()
plt.grid()
plt.show()
```

Output:

```
Weighted Sum: 0.46
Step Output: 1
Sigmoid Output: 0.613
ReLU Output: 0.46
Epoch 1 Errors: 33 Weights: [0.253 0.199] Bias: -0.3
Epoch 2 Errors: 23 Weights: [0.384 0.207] Bias: -0.4
Epoch 3 Errors: 14 Weights: [0.389 0.251] Bias: -0.4
Epoch 4 Errors: 14 Weights: [0.372 0.363] Bias: -0.4
Epoch 5 Errors: 21 Weights: [0.401 0.36 ] Bias: -0.5
Epoch 6 Errors: 12 Weights: [0.386 0.374] Bias: -0.5
Epoch 7 Errors: 14 Weights: [0.404 0.345] Bias: -0.5
Epoch 8 Errors: 12 Weights: [0.389 0.359] Bias: -0.5
Epoch 9 Errors: 12 Weights: [0.387 0.363] Bias: -0.5
Epoch 10 Errors: 12 Weights: [0.372 0.377] Bias: -0.5
Epoch 11 Errors: 12 Weights: [0.366 0.391] Bias: -0.5
Epoch 12 Errors: 14 Weights: [0.387 0.37 ] Bias: -0.5
Epoch 13 Errors: 12 Weights: [0.372 0.384] Bias: -0.5
Epoch 14 Errors: 14 Weights: [0.389 0.355] Bias: -0.5
Epoch 15 Errors: 12 Weights: [0.387 0.358] Bias: -0.5
Epoch 16 Errors: 12 Weights: [0.385 0.362] Bias: -0.5
Epoch 17 Errors: 12 Weights: [0.384 0.365] Bias: -0.5
Epoch 18 Errors: 12 Weights: [0.382 0.369] Bias: -0.5
Epoch 19 Errors: 12 Weights: [0.367 0.383] Bias: -0.5
Epoch 20 Errors: 14 Weights: [0.387 0.362] Bias: -0.5
```



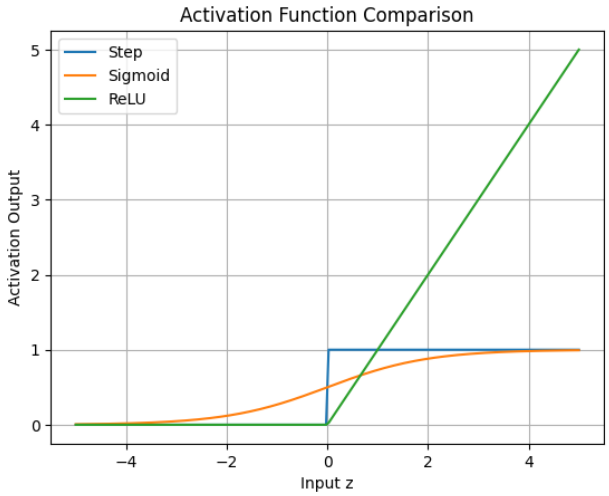


XOR Actual: [0 1 1 0]

XOR Predicted: [1 1 0 0]

XOR Correct: 2 out of 4

XOR converged: False



Result:
The perceptron was implemented successfully, and its ability to classify linearly separable data and its limitation in solving XOR were demonstrated.

Experiment 8: Two-Layer Neural Network Using Backpropagation

Aim:
To implement a two-layer neural network using forward propagation and backpropagation to solve the XOR classification problem and study error reduction during training.

Algorithm:

1. Prepare the input and target data for XOR.
2. Initialize weights and biases for the hidden and output layers.

3. Perform forward propagation to calculate the predicted output.
4. Calculate the loss between predicted and actual outputs.
5. Use backpropagation to calculate gradients.
6. Update weights and biases repeatedly.
7. Check the final predictions and plot the training loss.

Code:

```
#8) Two-Layer Neural Network with Backpropagation


import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z))
def forward(X, W1, b1, W2, b2):
    Z1 = X @ W1 + b1
    A1 = sigmoid(Z1)
    Z2 = A1 @ W2 + b2
    A2 = sigmoid(Z2)
    return Z1, A1, Z2, A2
def calculate_loss(A2, y):
    return -np.mean(
        y * np.log(A2 + 1e-8) +
        (1 - y) * np.log(1 - A2 + 1e-8)
    )
def backpropagation(X, y, W1, b1, W2, b2):
    n = len(X)
    Z1, A1, Z2, A2 = forward(X, W1, b1, W2, b2)
    dZ2 = (A2 - y) / n
    dW2 = A1.T @ dZ2
```

```
    db2 = np.sum(dZ2, axis=0, keepdims=True)
    dZ1 = (dZ2 @ W2.T) * A1 * (1 - A1)
    dW1 = X.T @ dZ1
    db1 = np.sum(dZ1, axis=0, keepdims=True)
    return dW1, db1, dW2, db2, A2
# Input: XOR dataset
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)
y = np.array([[0], [1], [1], [0]], dtype=float)
# Initialise weights and biases
W1 = np.random.randn(2, 8) * 0.5
b1 = np.zeros((1, 8))
W2 = np.random.randn(8, 1) * 0.5
b2 = np.zeros((1, 1))
print("INPUT DATA:")
print(X)
print("EXPECTED OUTPUT:")
print(y.ravel())
# Numerical gradient check before training
dW1, db1, dW2, db2, A2 = backpropagation(
    X, y, W1, b1, W2, b2
)
epsilon = 1e-5
original = W1[0, 0]
W1[0, 0] = original + epsilon
_, _, _, A_plus = forward(X, W1, b1, W2, b2)
```

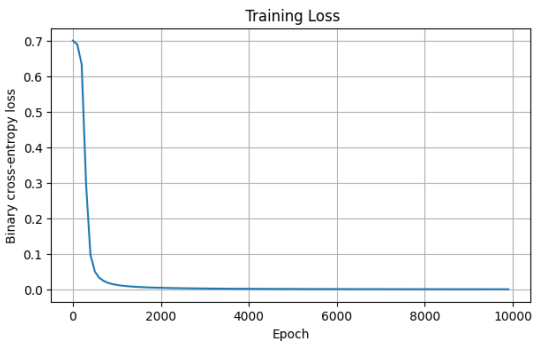
```
loss_plus = calculate_loss(A_plus, y)
W1[0, 0] = original - epsilon
_, _, _, A_minus = forward(X, W1, b1, W2, b2)
loss_minus = calculate_loss(A_minus, y)
W1[0, 0] = original
numerical_gradient = (loss_plus - loss_minus) / (2 * epsilon)
analytic_gradient = dW1[0, 0]
print("\nGRADIENT CHECK:")
print("Numerical gradient:", round(float(numerical_gradient), 6))
print("Analytical gradient:", round(float(analytic_gradient), 6))
print("Gradient check passed:",
      np.isclose(numerical_gradient, analytic_gradient,
                 rtol=1e-3, atol=1e-5))
# Train using gradient descent and backpropagation
epochs = 10000
learning_rate = 1.0
losses = []
for epoch in range(epochs):
    dW1, db1, dW2, db2, A2 = backpropagation(
        X, y, W1, b1, W2, b2
    )
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2
    if epoch % 100 == 0:
        _, _, _, A2 = forward(X, W1, b1, W2, b2)
        losses.append(calculate_loss(A2, y))
# Final predictions
_, _, _, probabilities = forward(X, W1, b1, W2, b2)
```

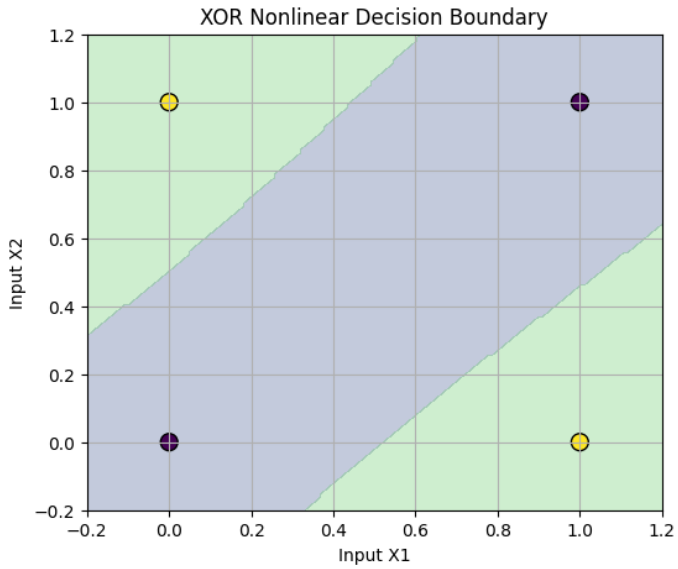
```
predictions = (probabilities >= 0.5).astype(int)
print("\nTRAINING RESULTS:")
print("Epochs:", epochs)
print("Final loss:", round(float(calculate_loss(probabilities, y)), 6))
print("Predicted probabilities:")
print(np.round(probabilities.ravel(), 4))
print("Predicted classes:", predictions.ravel())
print("Actual classes:", y.ravel())
print("Accuracy:",
     round(float(np.mean(predictions == y) * 100), 2), "%")
# Plot training loss
plt.figure(figsize=(7, 4))
plt.plot(np.arange(len(losses)) * 100, losses)
plt.xlabel("Epoch")
plt.ylabel("Binary cross-entropy loss")
plt.title(" Training Loss")
plt.grid(True)
plt.show()
# Plot nonlinear decision boundary
xx, yy = np.meshgrid(
   np.linspace(-0.2, 1.2, 150),
   np.linspace(-0.2, 1.2, 150))
grid = np.c_[xx.ravel(), yy.ravel()]
_, _, _, grid_probs = forward(grid, W1, b1, W2, b2)
Z = (grid_probs >= 0.5).astype(int).reshape(xx.shape)
plt.figure(figsize=(6, 5))
plt.contourf(xx, yy, Z, alpha=0.3, levels=[-0.5, 0.5, 1.5])
plt.scatter(X[:, 0], X[:, 1], c=y.ravel(),
          edgecolors="black", s=100)
plt.xlabel("Input X1")
```

```
plt.ylabel("Input X2")
plt.title("XOR Nonlinear Decision Boundary")
plt.grid(True)
plt.show()
```

Output:

```
INPUT DATA:
[[0. 0.]
 [0. 1.]
 [1. 0.]
 [1. 1.]]
EXPECTED OUTPUT:
[0. 1. 1. 0.]
GRADIENT CHECK:
Numerical gradient: 0.013816
Analytical gradient: 0.013816
Gradient check passed: True
TRAINING RESULTS:
Epochs: 10000
Final loss: 0.00067
Predicted probabilities:
[5.000e-04 9.994e-01 9.992e-01 8.000e-04]
Predicted classes: [0 1 1 0]
Actual classes: [0. 1. 1. 0.]
Accuracy: 100.0 %
```





Result:
The two-layer neural network was trained successfully using backpropagation and learned the XOR input-output relationships.

Experiment 9: Multilayer Perceptron (MLP)

Aim:
To build and train a Multilayer Perceptron for classification and analyse the effect of hidden layers, activation functions, and model performance.

Algorithm:

1. Load or generate a classification dataset.
2. Split the dataset into training and testing sets.
3. Create an MLP with one or more hidden layers.
4. Train the model using a suitable optimizer.
5. Predict the class labels for test data.
6. Evaluate accuracy and other classification metrics.
7. Compare the model's predictions with actual labels.

Code:

#9) Multilayer Perceptron Implementation

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt
```

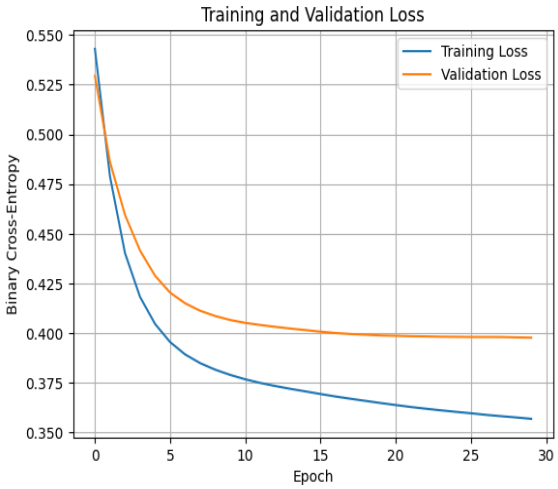
```
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, ConfusionMatrixDisplay
)
np.random.seed(42)
tf.random.set_seed(42)
# Generate synthetic telecom churn data
n = 1000
df = pd.DataFrame({
    "tenure": np.random.randint(1, 73, n),
    "monthly_charges": np.random.uniform(20, 120, n),
    "contract": np.random.choice(["Month-to-month", "One-year", "Two-year"], n),
    "complaints": np.random.randint(0, 6, n),
    "data_usage": np.random.uniform(1, 100, n)
})
prob = 1 / (1 + np.exp(-(
    1.5 - 0.035 * df["tenure"]
    + 0.015 * df["monthly_charges"]
    + 0.35 * df["complaints"]
    - 0.012 * df["data_usage"]
    + 0.5 * (df["contract"] == "Month-to-month")
)))
df["Churn"] = (np.random.rand(n) < prob).astype(int)
# Task 1: Encode categorical values
X = pd.get_dummies(df.drop(columns="Churn"), columns=["contract"], dtype=int)
y = df["Churn"].to_numpy()
X_train, X_test, y_train, y_test = train_test_split(
```

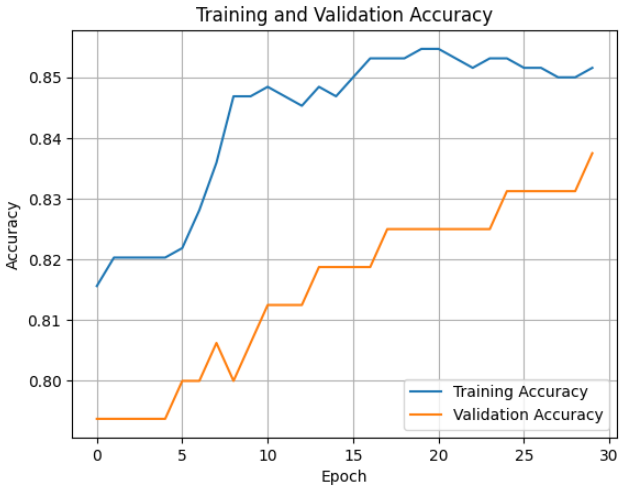
```
    X.to_numpy(), y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# Task 2: Build MLP with two hidden layers
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
# Task 3: Train with binary cross-entropy and Adam
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=30,
    batch_size=32,
    verbose=0
)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy")
plt.title("Training and Validation Loss")
```

```
plt.legend()
plt.grid()
plt.show()
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.grid()
plt.show()
# Task 4: Evaluation
pred = (model.predict(X_test, verbose=0).ravel() >= 0.5).astype(int)
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
print("Precision:", round(precision_score(y_test, pred, zero_division=0), 4))
print("Recall:", round(recall_score(y_test, pred, zero_division=0), 4))
print("F1 Score:", round(f1_score(y_test, pred, zero_division=0), 4))
cm = confusion_matrix(y_test, pred)
print("Confusion Matrix:\n", cm)
ConfusionMatrixDisplay(cm, display_labels=["No Churn", "Churn"]).plot()
plt.title("Customer Churn Confusion Matrix")
plt.show()
# Task 5: Compare hidden-layer widths
for width in [8, 16, 32, 64]:
    tf.keras.utils.set_random_seed(42)
    test_model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(X_train.shape[1],)),
        tf.keras.layers.Dense(width, activation="relu"),
        tf.keras.layers.Dense(1, activation="sigmoid")
    ])
```

```
    test_model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    test_model.fit(
        X_train, y_train, epochs=15, batch_size=32, verbose=0
    )
    score = test_model.evaluate(X_test, y_test, verbose=0)
    print("Width:", width, "Test Loss:", round(score[0], 4),
          "Test Accuracy:", round(score[1], 4))
```

Output:





Accuracy: 0.81

Precision: 0.8492

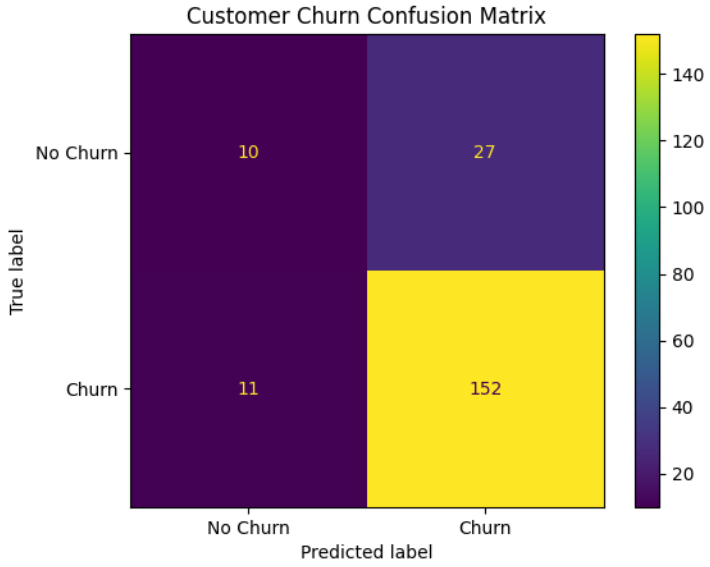
Recall: 0.9325

F1 Score: 0.8889

Confusion Matrix:

[[10 27]

[11 152]]



Width: 8 Test Loss: 0.4695 Test Accuracy: 0.815

Width: 16 Test Loss: 0.454 Test Accuracy: 0.82

Width: 32 Test Loss: 0.4455 Test Accuracy: 0.8

Width: 64 Test Loss: 0.4565 Test Accuracy: 0.8

Result:
The Multilayer Perceptron was trained and evaluated successfully, and its classification performance was analysed.

Experiment 10: L2 Regularization, Dropout and Early Stopping

Aim:
To apply L2 regularization, dropout, and early stopping to a neural network and analyse how these techniques help reduce overfitting.

Algorithm:

1. Prepare the training and validation datasets.
2. Build a neural network without regularization.
3. Train the model and record training and validation losses.
4. Apply L2 regularization, dropout, and early stopping.
5. Train the modified models.
6. Compare their training and validation performance.
7. Identify the techniques that improve generalization.

Code:

```
#10) L2, Dropout and Early Stopping Comparison


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
np.random.seed(42)
tf.random.set_seed(42)
# Generate 500 illustrative clinical records
X, y = make_classification(
    n_samples=500, n_features=10, n_informative=6,
    n_redundant=2, class_sep=0.8, flip_y=0.05,
    random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
X_fit, X_val, y_fit, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42,
    stratify=y_train
)
def build_model(l2=0, dropout=0):
    reg = tf.keras.regularizers.l2(l2) if l2 else None
```

```
    layers = [
        tf.keras.layers.Input(shape=(X_train.shape[1],)),
        tf.keras.layers.Dense(256, activation="relu",
                              kernel_regularizer=reg),
        tf.keras.layers.Dense(256, activation="relu",
                              kernel_regularizer=reg)
    ]
    if dropout:
        layers.append(tf.keras.layers.Dropout(dropout))
    layers.extend([
        tf.keras.layers.Dense(128, activation="relu",
                              kernel_regularizer=reg),
        tf.keras.layers.Dense(1, activation="sigmoid")
    ])
    model = tf.keras.Sequential(layers)
    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model
results = []
histories = {}
def evaluate_model(name, model, history):
    train_acc = model.evaluate(X_fit, y_fit, verbose=0)[1]
    test_acc = model.evaluate(X_test, y_test, verbose=0)[1]
    results.append({
        "Model": name,
        "Train Accuracy": train_acc,
        "Test Accuracy": test_acc,
```

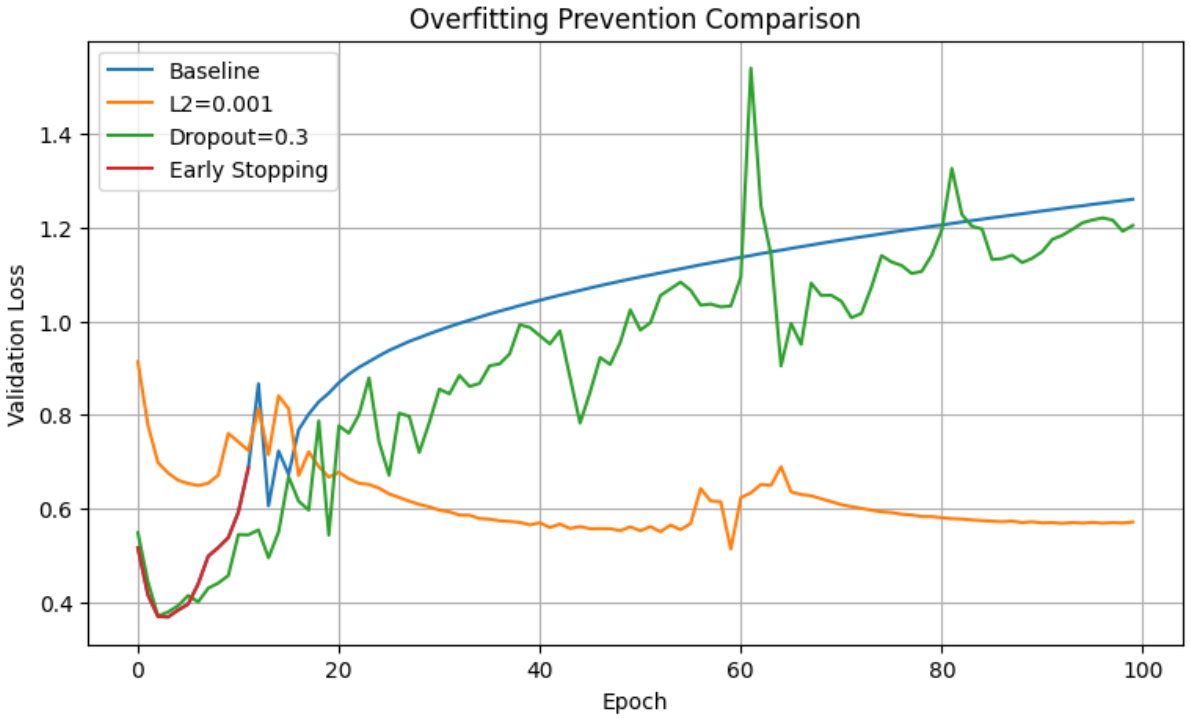
```
        "Train-Test Gap": train_acc - test_acc
    })
    histories[name] = history.history
# Task 1: Large baseline MLP
tf.keras.utils.set_random_seed(42)
model = build_model()
history = model.fit(
    X_fit, y_fit, validation_data=(X_val, y_val),
    epochs=100, batch_size=16, verbose=0
)
evaluate_model("Baseline", model, history)
# Task 2: L2 regularisation at three lambda values
for lam in [0.0001, 0.001, 0.01]:
    tf.keras.utils.set_random_seed(42)
    model = build_model(l2=lam)
    history = model.fit(
        X_fit, y_fit, validation_data=(X_val, y_val),
        epochs=100, batch_size=16, verbose=0
    )
    evaluate_model("L2=" + str(lam), model, history)
# Task 3: Dropout at three rates
for rate in [0.2, 0.3, 0.5]:
    tf.keras.utils.set_random_seed(42)
    model = build_model(dropout=rate)
    history = model.fit(
        X_fit, y_fit, validation_data=(X_val, y_val),
        epochs=100, batch_size=16, verbose=0
    )
    evaluate_model("Dropout=" + str(rate), model, history)
# Task 4: Early stopping
```

```
tf.keras.utils.set_random_seed(42)
model = build_model()
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=8, restore_best_weights=True
)
history = model.fit(
    X_fit, y_fit, validation_data=(X_val, y_val),
    epochs=100, batch_size=16, callbacks=[early_stop], verbose=0
)
evaluate_model("Early Stopping", model, history)
print(pd.DataFrame(results).round(4).to_string(index=False))
# Plot baseline and regularised validation losses
plt.figure(figsize=(9, 5))
for name in ["Baseline", "L2=0.001", "Dropout=0.3", "Early Stopping"]:
    plt.plot(histories[name]["val_loss"], label=name)
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.title("Overfitting Prevention Comparison")
plt.legend()
plt.grid()
plt.show()
```

Output:

Model	Train Accuracy	Test Accuracy	Train-Test Gap
Baseline	1.000	0.81	0.190
L2=0.0001	1.000	0.82	0.180
L2=0.001	1.000	0.82	0.180
L2=0.01	1.000	0.82	0.180
Dropout=0.2	1.000	0.82	0.180
Dropout=0.3	1.000	0.79	0.210
Dropout=0.5	1.000	0.80	0.200

Early Stopping	0.975	0.85	0.125



Result:

L2 regularization, dropout, and early stopping were implemented and compared to understand their effects on overfitting and generalization.